

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)
Assessment Tool Weightage: 35%

Faculty Name: Dr.G.Ayyappan

Student Reg.No: 192571049
Name: Hemalakshmi H

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	5
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	5
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	5
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	5
5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	5
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	5
7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	5

8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	5
9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	5
10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	5

Code of Conduct:

I Hemalakshmi H - 192571049 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. Pseudocode: Complete Query Processing Pipeline

Code

```
function QueryProcessingPipeline(SQL_Query):  
  
    // 1. Parsing  
    parseTree = parse(SQL_Query)  
    if syntaxError(parseTree):  
        return "Invalid Query"  
  
    // 2. Query Rewrite  
    algebraTree = convertToRelationalAlgebra(parseTree)  
    algebraTree = applyHeuristicOptimizations(algebraTree)  
  
    // 3. Cost-Based Optimization  
    bestPlan = generateCandidatePlans(algebraTree)  
    bestPlan = chooseLowestCostPlan(bestPlan)  
  
    // 4. Execution Plan Generation  
    execPlan = translateToPhysicalOperators(bestPlan)  
  
    // 5. Execution  
    result = executePlan(execPlan)  
  
    // 6. Output  
    return formatResult(result)
```

2. Pseudocode: Cost-Based Optimization for Join Ordering

Code

```
function CostBasedJoinOrdering(relations):  
  
    candidateOrders = generateAllJoinOrders(relations)  
    bestCost = infinity
```

```
bestOrder = null
```

```
for order in candidateOrders:
```

```
    cost = 0
```

```
    for each joinStep in order:
```

```
        cost += estimateIOCost(joinStep)
```

```
    if cost < bestCost:
```

```
        bestCost = cost
```

```
        bestOrder = order
```

```
return bestOrder
```

Cost estimation example: $\text{estimateIOCost} = \text{pages}(R) + \text{pages}(S)$ for hash join $\text{pages}(R) + \text{tuples}(R) * \text{pages}(S)$ for nested loop

3. Pseudocode: Nested Loop Join + Time Complexity

Code

```
function NestedLoopJoin(R, S, joinCondition):
```

```
    result = emptySet
```

```
    for each tuple r in R:
```

```
        for each tuple s in S:
```

```
            if joinCondition(r, s):
```

```
                add (r, s) to result
```

```
return result
```

Time Complexity

- Outer loop: $|R|$ tuples
- Inner loop: $|S|$ tuples
- Total comparisons: $|R| \times |S|$

If pages are used: $\text{Cost} \approx \text{pages}(R) + |R| * \text{pages}(S)$ (block nested loop)

4. Pseudocode: Deadlock Detection Using Wait-For Graph

Code

```
function DetectDeadlock(lockTable):
```

```
    WFG = buildWaitForGraph(lockTable)
```

```
    visited = emptySet
```

```
    recursionStack = emptySet
```

```
    for each transaction T in WFG:
```

```
        if DFS(T, visited, recursionStack):
```

```
            return "Deadlock Detected"
```

```
    return "No Deadlock"
```

```
function DFS(T, visited, recursionStack):
```

```
    if T not in visited:
```

```
        add T to visited
```

```
        add T to recursionStack
```

```
    for each neighbor N in WFG[T]:
```

```
        if N not in visited and DFS(N, visited, recursionStack):
```

```
            return true
```

```
        else if N in recursionStack:
```

```
            return true
```

```
    remove T from recursionStack
```

```
    return false
```

5. Pseudocode: Basic Two-Phase Locking (2PL)

Code

```
function TwoPhaseLocking(T, operations):
```

```
    phase = "growing"
```

```
    for op in operations:
```

```
        if op is READ or WRITE:
```

```
            if phase == "shrinking":
```

```
                abort(T)
```

```
            if lockNeeded(op):
```

```
                if lockAvailable(op):
```

```
                    grantLock(T, op)
```

```
                else:
```

```
                    wait(T)
```

```
            else if op is UNLOCK:
```

```
                releaseLock(T, op)
```

```
                phase = "shrinking"
```

```
    commit(T)
```

6. Pseudocode: Timestamp-Based Concurrency Control (Thomas' Write Rule)

Code

```
function TimestampCC(operation, TS):
```

```
    if operation is READ(X) by Ti:
```

```
        if  $TS(T_i) < WTS(X)$ :
```

```
            abort(Ti)
```

```
        else:
```

```
             $RTS(X) = \max(RTS(X), TS(T_i))$ 
```

```
            return X
```

if operation is WRITE(X) by Ti:

if $TS(Ti) < RTS(X)$:

 // Thomas' Write Rule

 ignoreWrite(Ti, X)

else if $TS(Ti) < WTS(X)$:

 abort(Ti)

else:

$WTS(X) = TS(Ti)$

 performWrite(Ti, X)

7. Pseudocode: ARIES Recovery (Analysis, Redo, Undo)

Code

function ARIES_Recovery(log):

 // Phase 1: Analysis

 dirtyPages = {}

 activeTransactions = {}

 for each entry in log:

 updateTransactionStatus(entry, activeTransactions)

 updateDirtyPages(entry, dirtyPages)

 // Phase 2: Redo

 for each entry in log from lastCheckpoint:

 if entry affects dirtyPages:

 redo(entry)

 // Phase 3: Undo

 for each T in activeTransactions:

 for each logEntry of T in reverse:

 undo(logEntry)

 writeCompensationLogRecord(logEntry)

```
return "Recovery Complete"
```

8. Pseudocode: Shadow Paging Recovery

Code

```
function ShadowPaging():
```

```
    shadowPageTable = copy(currentPageTable)
```

```
    try:
```

```
        performTransactionUpdates(currentPageTable)
```

```
        commit()
```

```
        shadowPageTable = currentPageTable
```

```
    except failure:
```

```
        currentPageTable = shadowPageTable
```

```
        rollback()
```

9. Pseudocode: Immediate Update Recovery (UNDO/REDO)

Code

```
function ImmediateUpdateRecovery(log):
```

```
    committed = findCommittedTransactions(log)
```

```
    uncommitted = findUncommittedTransactions(log)
```

```
    // REDO committed
```

```
    for each entry in log:
```

```
        if entry.T in committed:
```

```
            redo(entry)
```

```
    // UNDO uncommitted
```

```
    for each entry in reverse(log):
```

```
        if entry.T in uncommitted:
```

```
undo(entry)
```

```
return "Recovery Complete"
```

10. Pseudocode: Database Connectivity Manager (Pooling + Execution + Rollback)

Code

```
class DBConnectionManager:
```

```
    pool = initializePool(maxConnections)
```

```
    function getConnection():
```

```
        if pool has free connection:
```

```
            return pool.get()
```

```
        else:
```

```
            wait()
```

```
    function executeQuery(query):
```

```
        con = getConnection()
```

```
        try:
```

```
            result = con.execute(query)
```

```
            con.commit()
```

```
            pool.release(con)
```

```
            return result
```

```
        except Exception:
```

```
            con.rollback()
```

```
            pool.release(con)
```

```
            return "Error"
```

```
    function closeAll():
```

```
        for con in pool:
```

```
            con.close()
```